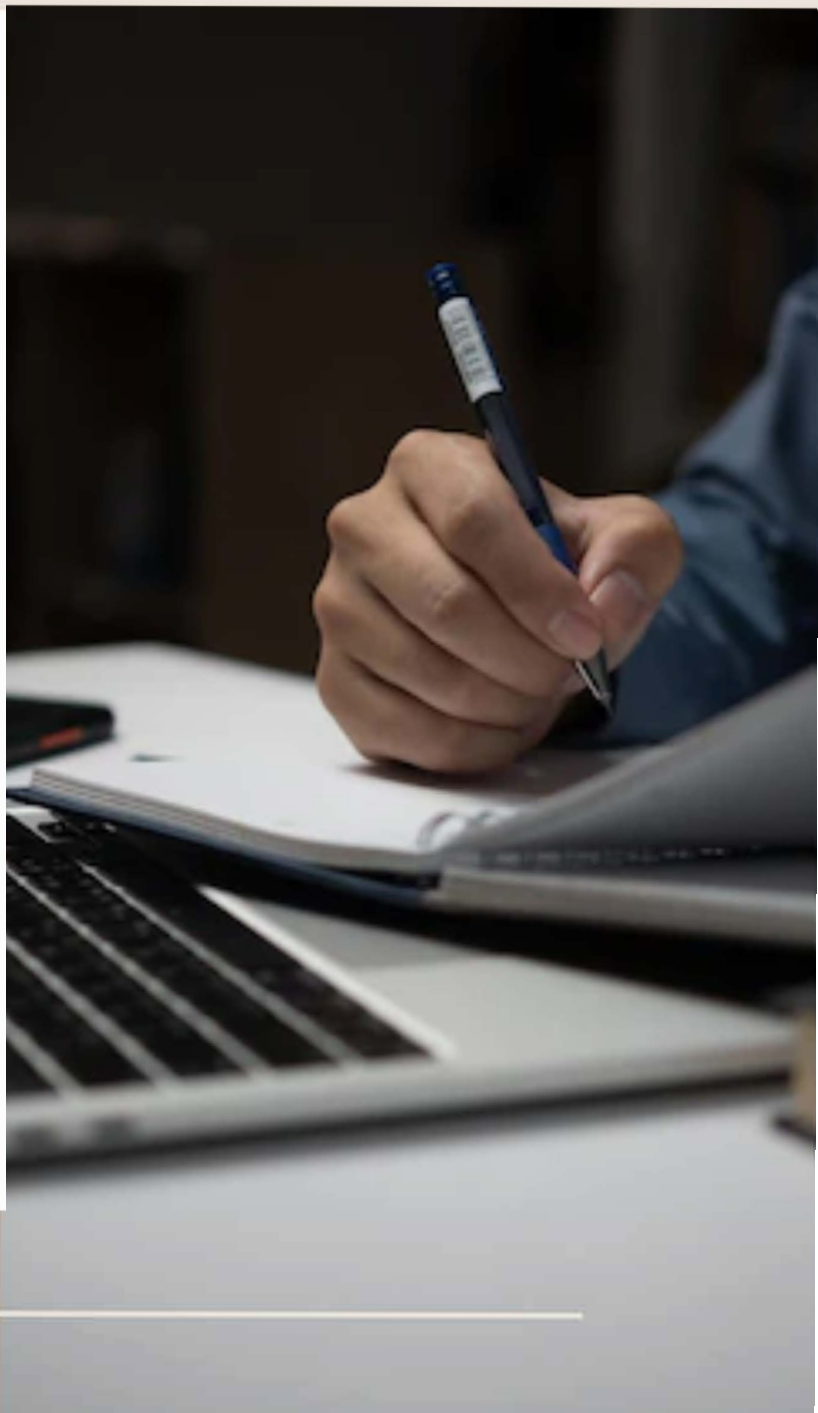


PROB. AND STAT. LAB FILE

Submitted by:-
Nishant Kanwar
Roll no. 3024030021
Msc. maths and computing

2025



ASSIGNMENT 1

Q1

```
seq(from = 1, to = 20, by = 1)
```

OUTPUT:

```
# [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
```

Q2

```
X <- c(10,20,30,40,50,60)
```

```
max(X)
```

```
min(X)
```

OUTPUT:

```
# max(X) -> [1] 60
```

```
# min(X) -> [1] 10
```

Q3

```
x <- seq(from = 0, to = 2*pi, by = 0.01)
```

```
y <- sin(x)
```

```
plot(x, y, type = "l", col = "green", xlab = "X-Axis", ylab = "Y-Axis", main = "Sine function")
```

OUTPUT: (Sine function plot)

Q4

```
num <- as.integer(readline(prompt = "Enter a number: "))
```

```
if (num < 0) {
```

```
  cat("Warning : Negative Number")
```

```
} else {
```

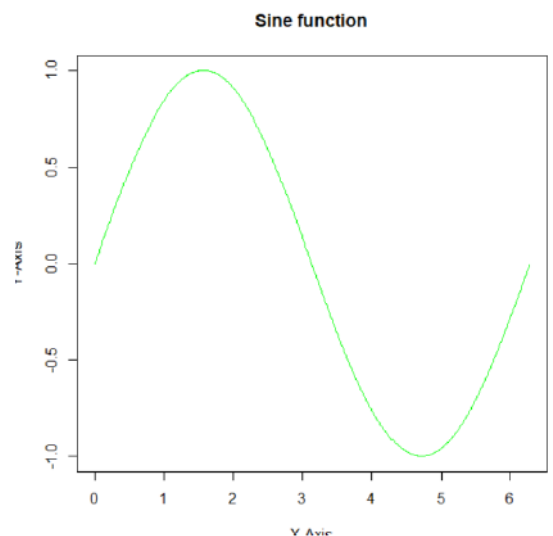
```
  fact <- factorial(num)
```

```
  cat("Factorial of", num, "is", fact, "\n")
```

```
}
```

Example OUTPUT (when input 4):

Factorial of 4 is 24



Q5

```
x <- c(1,2,3,4)
```

```
mean(x)
```

OUTPUT:

```
# [1] 2.5
```

Q6

```
num1 <- as.integer(readline(prompt = "Enter a number: "))
```

```
x <- 1
```

```
y <- 1
```

```
cat(x, "\n")
```

```
cat(y, "\n")
```

```
if(num1 < 0){
```

```
  print("Error , Negative Input")
```

```
} else {
```

```

if(num1 >= 3){
  for(i in 3:num1){
    fib <- x + y
    x <- y
    y <- fib
    cat(fib, "\n")
  }
}
}
}
# OUTPUT: (Fibonacci sequence printed up to n terms)

```

Q7

```

num1 <- as.numeric(readline(prompt = "Enter first number : "))
num2 <- as.numeric(readline(prompt = "Enter second number : "))
op <- readline(prompt = "Enter operator : ")
if(op == "+"){
  result <- num1 + num2
} else if(op == "-"){
  result <- num1 - num2
} else if(op == "*"){
  result <- num1 * num2
} else if(op == "/"){
  if(num2 == 0){
    result <- "Error : Division by zero"
  } else {
    result <- num1 / num2
  }
} else {
  result <- "Invalid operator"
}
cat("Result : ", result, "\n")
# Example INPUT: 12, 3, /
# OUTPUT: Result : 4

```

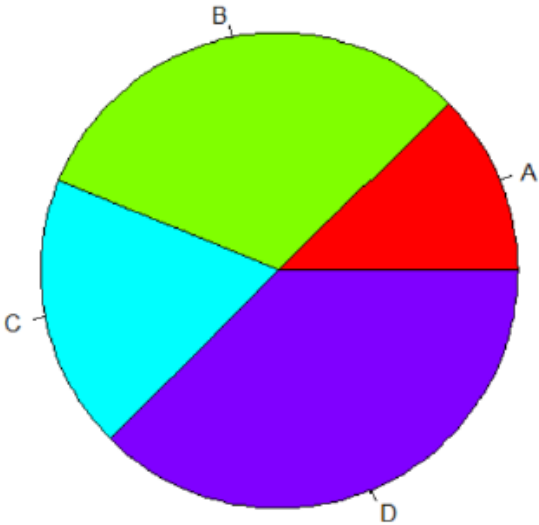
Q8

```

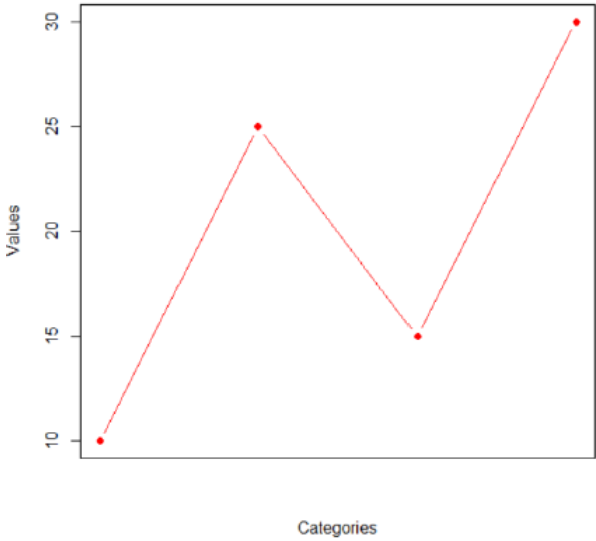
categories <- c("A","B","C","D")
values <- c(10,25,15,30)
pie(values, labels = categories, main = "Sample pie", col = rainbow(length(values)))
barplot(values, names.arg = categories, main = "Sample Bar Graph", col = "skyblue", ylab = "Values")
plot(values, type = "b", main = "Sample Line Graph", xlab = "Categories", ylab = "Values", xaxt = "n", col = "red", pch = 19)
# Sample scatter plot
x <- c(5, 10, 15, 20, 25)
y <- c(7, 12, 18, 24, 30)
plot(x, y, main = "Sample Scatter Plot", xlab = "X Values", ylab = "Y Values", pch = 19, col = "blue")
# OUTPUT: (Pie, bar, line, and scatter plots displayed)

```

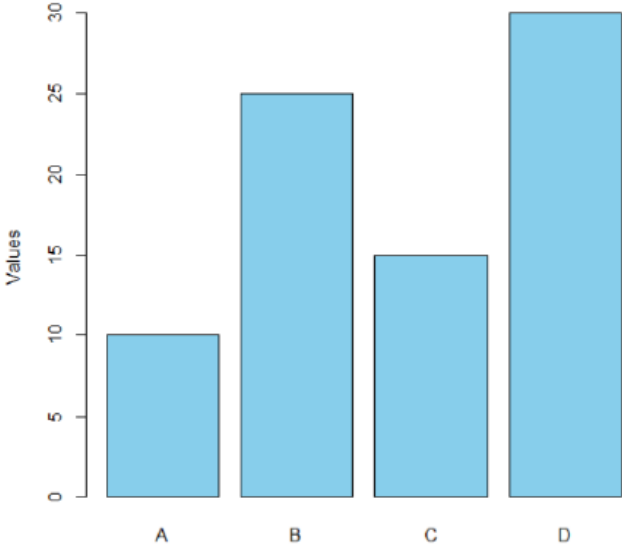
Sample pie



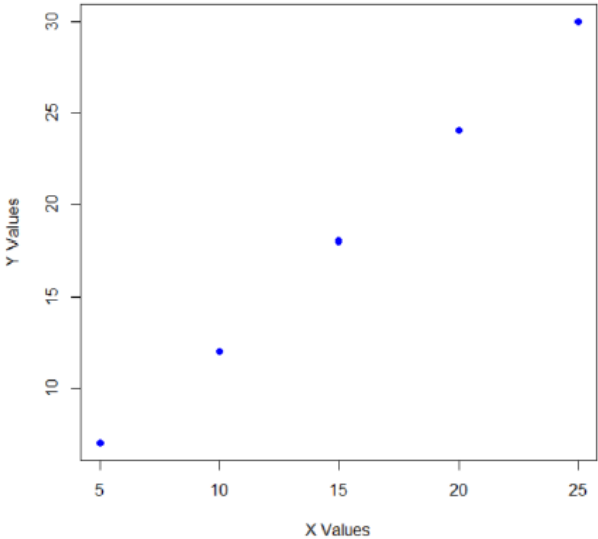
Sample Line Graph



Sample Bar Graph



Sample Scatter Plot



ASSIGNMENT 2

Q1

```
data(iris)
head(iris)
diff(range(iris$Sepal.Length))
diff(range(iris$Sepal.Width))
diff(range(iris$Petal.Length))
diff(range(iris$Petal.Width))
mean(iris$Petal.Width)
mean(iris$Sepal.Length)
mean(iris$Sepal.Width)
mean(iris$Petal.Length)
median(iris$Petal.Width)
median(iris$Sepal.Length)
median(iris$Sepal.Width)
median(iris$Petal.Length)
quantile(iris$Sepal.Length, 0.25)
quantile(iris$Sepal.Length, 0.75)
IQR(iris$Sepal.Length)
quantile(iris$Petal.Length, 0.25)
quantile(iris$Petal.Length, 0.75)
IQR(iris$Sepal.Length)
quantile(iris$Sepal.Width, 0.25)
quantile(iris$Sepal.Width, 0.75)
IQR(iris$Sepal.Length)
quantile(iris$Petal.Width, 0.25)
quantile(iris$Petal.Width, 0.75)
IQR(iris$Sepal.Length)
var(iris$Petal.Width)
var(iris$Sepal.Length)
var(iris$Sepal.Width)
var(iris$Petal.Length)
sd(iris$Petal.Width)
sd(iris$Sepal.Length)
sd(iris$Sepal.Width)
sd(iris$Petal.Length)
summary(iris)
# OUTPUT (excerpt):
# diff range Sepal.Length -> 3.6
# mean Sepal.Length -> 5.843333
# mean Sepal.Width -> 3.057333
# mean Petal.Length -> 3.758
# mean Petal.Width -> 1.199333
# summary shows min/1stQ/median/mean/3rdQ/max and species counts
```

Q2

```
mode <- function(x){
  freq <- table(x)
  mode_val <- names(freq)[freq == max(freq)]
  if(is.numeric(x)){
    mode_val <- as.numeric(mode_val)
```

```

}
return(mode_val)
}
x <- c(1, 2, 2, 3, 4, 4, 4, 5)
mode(x)
# OUTPUT:
# [1] 4

```

Q3

```

birthday_same <- function(M, trials = 10000) {
  matches <- 0
  for (i in 1:trials) {
    birthdays <- sample(1:365, M, replace = TRUE)
    if (any(duplicated(birthdays))) {
      matches <- matches + 1
    }
  }
  probability <- matches / trials
  return(probability)
}
birthday_same(10)
birthday_same(23)
M <- 1
prob <- 0
while (prob <= 0.5) {
  prob <- birthday_same(M)
  cat("M =", M, "Probability =", prob, "\n")
  M <- M + 1
}
# OUTPUT (examples):
# birthday_same(10) -> approx 0.1152
# birthday_same(23) -> approx 0.4967
# Smallest M with probability > 0.5: 23

```

Q4

```

rain_given_clouds <- function(pC, pR, pC_given_R) {
  prob <- (pC_given_R * pR) / pC
  return(prob)
}
rain_given_clouds(0.4, 0.2, 0.85)
# OUTPUT:
# [1] 0.425

```

Q5

```

urn_prob <- function(red, total) {
  p_U <- 1/3
  p_R_given_U <- red / total
  return(p_R_given_U * p_U)
}
denom <- urn_prob(8, 12) + urn_prob(6, 12) + urn_prob(5, 12)

```

```

cat("From I:", urn_prob(8, 12)/denom, "\n")
cat("From II:", urn_prob(6, 12)/denom, "\n")
cat("From III:", urn_prob(5, 12)/denom, "\n")
# OUTPUT:
# From I: 0.4210526
# From II: 0.3157895
# From III: 0.2631579

```

```

Q6
defective_car <- function() {
pA <- 2/5
pB <- 2/5
pC <- 1/5
dA <- 0.02
dB <- 0.03
dC <- 0.04
total_def <- dA*pA + dB*pB + dC*pC
p_A_given_def <- (dA*pA) / total_def
list(prob_defective = total_def, prob_A_given_def = p_A_given_def)
}
defective_car()
# OUTPUT:
# $prob_defective
# [1] 0.028
# $prob_A_given_def
# [1] 0.2857143

```

```

Q7
led_defect_prob <- function() {
pA <- 0.2; pB <- 0.3; pC <- 0.5
dA <- 0.01; dB <- 0.02; dC <- 0.03
total_def <- dA*pA + dB*pB + dC*pC
p_A_given_def <- (dA * pA) / total_def
p_B_given_def <- (dB * pB) / total_def
p_C_given_def <- (dC * pC) / total_def
list(
  prob_defective = total_def,
  prob_A_given_def = p_A_given_def,
  prob_B_given_def = p_B_given_def,
  prob_C_given_def = p_C_given_def
)
}
led_defect_prob()
# OUTPUT:
# $prob_defective
# [1] 0.023
# $prob_A_given_def
# [1] 0.08695652
# $prob_B_given_def
# [1] 0.2608696
# $prob_C_given_def
# [1] 0.6521739

```

ASSIGNMENT 3

Q1

```
x <- c(0,1,2,3,4)
y <- c(0.41,0.37,0.16,0.05,0.01)
EV1 <- sum(x^2 * y)
cat("EV1:", EV1, "\n")
EV2 <- sum((2*x + 3) * y)
cat("EV2:", EV2, "\n")
EV3 <- sum((2*x - 3) * y)
cat("EV3:", EV3, "\n")
EV4 <- sum((2*x - 3)^2 * y)
var <- EV4 - (EV3)^2
cat("variance :", var, "\n")
# OUTPUT:
# EV1: 1.62
# EV2: 4.76
# EV3: -1.24
# variance : 3.3824
```

Q2

```
x <- c(0, 1, 2, 3)
px <- c(0.1, 0.2, 0.2, 0.5)
y <- 24*x - 12*3 + 4*(3-x) # as per given expression
EV <- sum(y * px)
V <- sum((y)^2 * px) - (EV)^2
cat("EV:", EV, "\n")
cat("variance :", V, "\n")
# OUTPUT:
# EV: 18
# variance : 436
```

Q3

```
X <- c(-3, -1, 0, 1, 2, 3, 5, 8)
F <- c(0.10, 0.30, 0.45, 0.5, 0.75, 0.90, 0.95, 1.0)
pmf <- c(F[1], diff(F))
cat("PMF :", pmf, "\n")
even <- X %% 2 == 0
P_even <- sum(pmf[even])
P_1to8 <- sum(pmf[X >= 1 & X <= 8])
P_gt0 <- sum(pmf[X > 0])
P_ge3_and_gt0 <- sum(pmf[X >= 3])
prob <- P_ge3_and_gt0 / P_gt0
list(P_even = P_even, P_1to8 = P_1to8, P_Xge3_given_Xgt0 = prob)
# OUTPUT:
# PMF : 0.10 0.20 0.15 0.05 0.25 0.15 0.05 0.05
# P_even = 0.45
# P_1to8 = 0.55
```



```
# P_Xge3_given_Xgt0 = 0.4545455
```

Q4

```
f <- function(t) { ifelse(t >= 0, 0.2 * exp(-0.2 * t), 0) }
mean_T <- integrate(function(t) t * f(t), 0, Inf)$value
mean_T2 <- integrate(function(t) t^2 * f(t), 0, Inf)$value
var_T <- mean_T2 - mean_T^2
mean_T
var_T
# For 2T - 3
mean_2T3 <- 2 * mean_T - 3
var_2T3 <- 4 * var_T
mean_2T3
var_2T3
# OUTPUT:
# mean_T = 5
# var_T = 25
# mean_2T3 = 7
# var_2T3 = 100
```

Q5

```
f <- function(x) { ifelse(x >= 0, 3 * exp(-3 * x), 0) }
mean_x <- integrate(function(x) x * f(x), 0, Inf)$value
mean_x2 <- integrate(function(x) x^2 * f(x), 0, Inf)$value
var_x <- mean_x2 - mean_x^2
mean_2X3 <- 2 * mean_x + 3
var_2X3 <- 4 * var_x
mean_x
var_x
mean_2X3
var_2X3
# OUTPUT:
# mean_x = 0.3333333
# var_x = 0.1111111
# mean_2X3 = 3.666667
# var_2X3 = 0.4444444
```

Q6

```
f <- function(x) { ifelse(x > 1 & x < 20, 0.5 * exp(-0.5 * x), 0) }
mean_x <- integrate(function(x) x * f(x), 1, 20)$value
mean_x2 <- integrate(function(x) x^2 * f(x), 1, 20)$value
var_x <- mean_x2 - mean_x^2
mean_x
var_x
# OUTPUT:
# mean_x = 1.818593
# var_x = 4.555462
```

Q7

```

dice <- 1:6
outcomes <- expand.grid(dice, dice)
sums <- rowSums(outcomes)
mean_sums <- mean(sums)
var_sums <- var(sums)
mean_sums
var_sums
# OUTPUT:
# mean_sums = 7
# var_sums = 6

```

```

Q8
f <- function(x) { (3/4) * (1/4)^(x-1) }
x <- 1:10
y <- x^2
py <- f(x)
prob_Y_values <- py[1:5]
names(prob_Y_values) <- y[1:5]
P_Y3 <- f(3)
EY <- sum(y * py)
VarY <- sum((y - EY)^2 * py)
EY
VarY
prob_Y_values
P_Y3
# OUTPUT:
# EY = 2.222099
# VarY = 9.12025
# prob_Y_values for {1,4,9,16,25} = 0.75, 0.1875, 0.046875, 0.01171875, 0.002929688
# P_Y3 = 0.046875

```

ASSIGNMENT 4

```

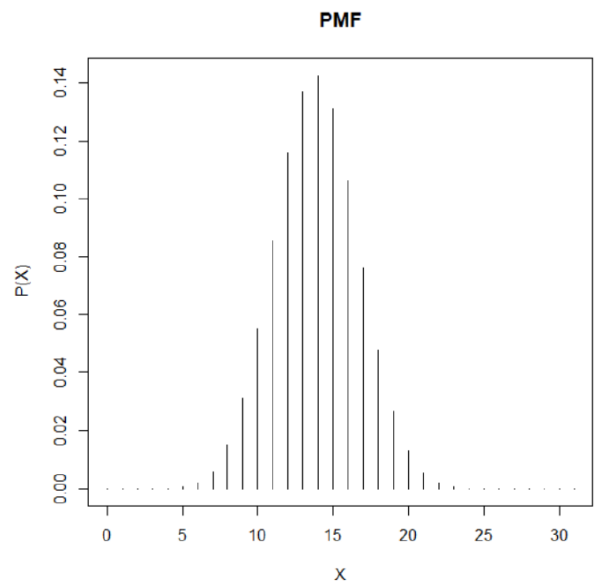
Q1
n <- 12
p <- 1/6
p_a <- dbinom(8, n, p)
p_b <- pbinom(6, n, p)
p_c <- 1 - pbinom(6, n, p)
p_d <- pbinom(9, n, p) - pbinom(6, n, p)
p_e <- pbinom(8, n, p) - pbinom(7, n, p)
p_a; p_b; p_c; p_d; p_e
# OUTPUT:
# p_a = 0.0001421249
# p_b = 0.9987075
# p_c = 0.001292544
# p_d = 0.001291758
# p_e = 0.0001421249

```

```

Q2
n <- 31
p <- 0.4477
pmf <- dbinom(0:n, n, p)
cdf <- pbinom(0:n, n, p)
mean_X <- n * p
var_X <- n * p * (1 - p)
sd_X <- sqrt(var_X)
mean_X; var_X; sd_X
# OUTPUT:
# mean_X = 13.8787
# var_X = 7.665206
# sd_X = 2.768611

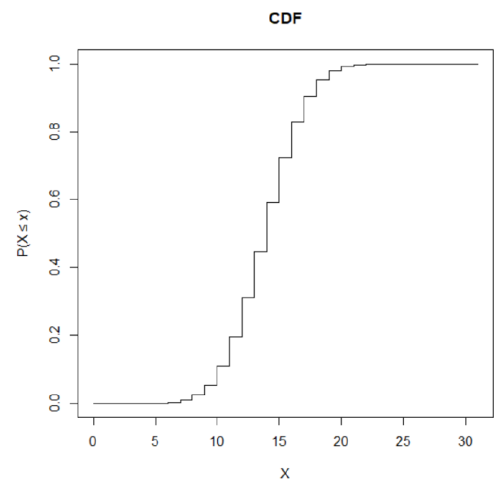
```



```

Q3
n <- 2000
p <- 0.001
p_a <- dbinom(3, n, p)
p_b <- 1 - pbinom(2, n, p)
p_a; p_b
# OUTPUT:
# p_a = 0.1805373
# p_b = 0.3233236

```



```

Q4
n <- 100000
p <- 2e-5
lambda <- n * p
p_event <- 1 - ppois(4, lambda)
p_event
# OUTPUT:
# 0.05265302

```

```

Q5
# (a)  $\lambda = 5$ , Prob no car arrives
ppois(0, lambda = 5)
# OUTPUT: 0.006737947

```

```

# (b) For 10 hours  $\lambda = 50$ ,  $P(48 \leq X \leq 50)$ 
p_48_50 <- ppois(50, lambda = 50) - ppois(47, lambda = 50)
p_48_50
# OUTPUT: 0.1678485

```

```

Q6
p <- 0.6
max_k <- 100
probs_odd <- dgeom(seq(0, max_k, by = 2), p)
p_odd <- sum(probs_odd)

```

```
p_odd  
# OUTPUT: 0.7142857
```

```
Q7  
prob <- 0.05  
# Prob 5th hit occurs on 10th throw (negative binomial)  
p_prob <- dnbinom(5 - 1, size = 5, prob = prob)  
p_prob  
# OUTPUT: 1.781732e-05
```

```
Q8  
r <- 2  
p_def <- 0.05  
# Prob at least 4 sampled to get 2 defective ->  $P(X \geq 4)$   
p_event <- 1 - pnbinom(3 - 2, size = r, prob = p_def)  
p_event  
# OUTPUT: 0.99275
```

ASSIGNMENT – 5

Ques 1

If X is uniformly distributed over the interval $[-2, 2]$ write an R program to find:

1. $P(X < 0)$
2. $P(|X - 1| \geq 1/2)$

```
a <- -2  
b <- 2
```

```
# 1.  $P(X < 0)$   
p1 <- punif(0, min = a, max = b)
```

```
# 2.  $P(|X - 1| \geq 1/2)$   
# This is  $X \leq -0.5$  OR  $X \geq 1.5$   
p2 <- punif(-0.5, min = a, max = b) + (1 - punif(1.5, min = a, max = b))
```

```
p1  
p2
```

Output: p1 = 0.5, p2 = 0.75

Ques 2

```
a <- 0  
b <- 60
```

```
# (i)  $P(X > 35)$   
p1 <- 1 - punif(35, min = a, max = b)
```

```
# (ii)  $P(10 \leq X \leq 25)$   
p2 <- punif(25, min = a, max = b) - punif(10, min = a, max = b)
```

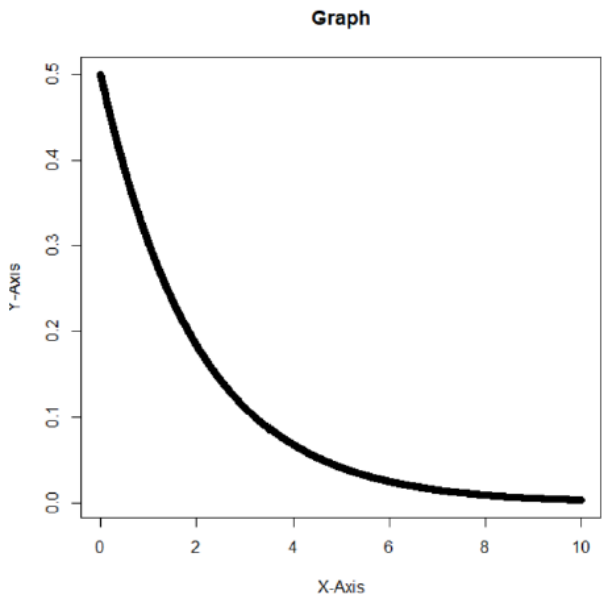
```
p1  
p2
```

Output: p1 = 0.4166667, p2 = 0.25

Q3

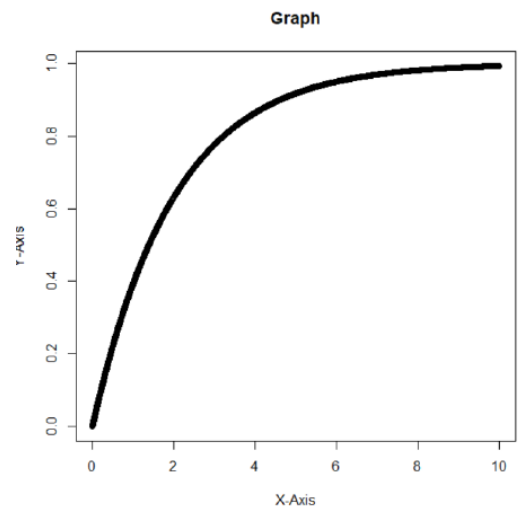
(i) `dexp(4, 0.5)`
Output: 0.06766764

(ii)
`x <- seq(0, 10, 0.001)`
`y <- dexp(x, 0.5)`
`plot(x, y, xlab = "X-Axis", ylab = "Y-Axis", main = "Graph")`



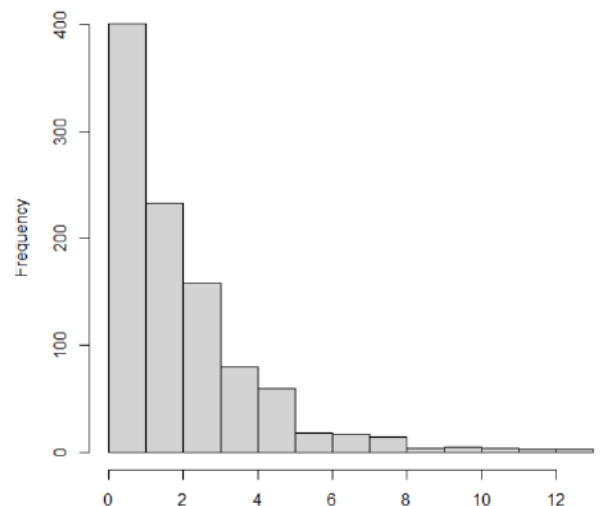
(iii) `pexp(4, 0.5)`
Output: 0.8646647

(iv)
`x <- seq(0, 10, 0.001)`
`y <- pexp(x, 0.5)`
`plot(x, y, xlab = "X-Axis", ylab = "Y-Axis", main = "Graph")`



(v)
`x <- rexp(1000, 0.5)`
`hist(x)`

Histogram of x



Ques 4-a
1 - `pexp(3, 2)`

Ques 4-b
`(1 - pexp(3, 2)) / (1 - pexp(2, 2))`

Output: 0.002478752
0.1353353

Ques 5-a
`300 * pnorm(57, 64.5, 5)`

Ques 5-b
`300 * (pnorm(72, 64.5, 5) - pnorm(57, 64.5, 5))`

Output: 20.04216
259.9157

Ques 6-a

```
1 - pgamma(1, 2, , 1/3)
```

Ques 6-b

```
qgamma(0.7, 2, , 1/3)
```

Output: 0.1991483

0.8130722

Ques 7

```
q1 <- 30
```

```
q2 <- 62
```

```
z1 <- qnorm(0.1)
```

```
z2 <- qnorm(0.97)
```

```
s <- (q2 - q1) / (z2 - z1)
```

```
s
```

```
mean <- q1 - s * z1
```

```
mean
```

Output: 10.11907

42.96811

ASSIGNMENT – 6

Q1

```
# Define x and y values
```

```
x <- 0:2
```

```
y <- 0:2
```

```
# Create joint pmf matrix
```

```
pmf <- outer(x, y, function(a, b) (2*a + b) / 27)
```

```
pmf
```

```
## (i) Display joint pmf
```

```
print("Joint PMF matrix:")
```

```
pmf
```

```
## (ii) Check if it is a valid pmf (sum should be 1)
```

```
total_sum <- sum(pmf)
```

```
total_sum
```

```
## (iii) Marginal distribution pX(x)
```

```
pX <- apply(pmf, 1, sum)
```

```
pX
```

```
## (iv) Marginal distribution pY(y)
```

```
pY <- apply(pmf, 2, sum)
```

```
pY
```

```
## (v) Conditional probability  $P(X=0 \mid Y=1)$ 
```

```
P_X0_given_Y1 <- pmf[1, 2] / pY[2] # pmf[x=0,y=1] / pY(1)
```

```
P_X0_given_Y1
```

```
## (vi) Expectations, variances, covariance, correlation
```

```
# Expected X
```

```
E_X <- sum(x * pX)
```

```
# Expected Y
```

```
E_Y <- sum(y * pY)
```

```
# Expected XY
```

```
E_XY <- sum(outer(x, y, "*") * pmf)
```

```
# Var(X)
```

```
E_X2 <- sum((x^2) * pX)
```

```
Var_X <- E_X2 - (E_X)^2
```

```
# Var(Y)
```

```
E_Y2 <- sum((y^2) * pY)
```

```
Var_Y <- E_Y2 - (E_Y)^2
```

```
# Cov(X,Y)
```

```
Cov_XY <- E_XY - (E_X * E_Y)
```

```
# Correlation coefficient
```

```
rho <- Cov_XY / sqrt(Var_X * Var_Y)
```

```
# Display all
```

```
E_X; E_Y; E_XY
```

```
Var_X; Var_Y
```

```
Cov_XY
```

```
rho
```



```

Q2 # Joint PDF
f <- function(x, y) {
  ifelse(x >= 0 & x <= 1 & y >= 0 & y <= 1,
    (2/5) * (x + 4*y),
    0)
}

# (i) Check if valid PDF:  $\int_0^1 \int_0^1 f(x,y) dx dy$ 
total_integral <- integral2(f, xmin = 0, xmax = 1, ymin = 0, ymax = 1)
total_integral

# (ii) Marginal  $f_X(x) = \int_0^1 f(x,y) dy$  evaluated at  $x = 1$ 
fX <- function(x) {
  sapply(x, function(xx)
    integrate(function(yy) f(xx, yy), 0, 1)$value)
}

fX_1 <- fX(1)
fX_1

# (iii) Marginal  $f_Y(y) = \int_0^1 f(x,y) dx$  evaluated at  $y = 0$ 
fY <- function(y) {
  sapply(y, function(yy)
    integrate(function(xx) f(xx, yy), 0, 1)$value)
}

fY_0 <- fY(0)
fY_0

# (iv) Expected value  $E[g(X,Y)] = E(XY)$ 
E_XY <- integral2(
  function(x, y) x * y * f(x, y),
  xmin = 0, xmax = 1,
  ymin = 0, ymax = 1
)

E_XY

```

ASSIGNMENT – 7

Ques 1

The average height of residents is 168 cm with $\sigma = 3.9$ cm.
 Doctor measured 36 individuals and found mean = 169.5 cm.

- State Null and Alternate Hypothesis
- At 95% CI, test if Null can be rejected.

```
h0 = 168
n = 36
s = 3.9
m = 169.5
cl = 0.95
```

```
z = (m - h0) / (s / sqrt(n))
p = 2 * (1 - pnorm(abs(z)))
```

```
if(p > 0.05) cat("accepted")
else cat("rejected")
```

Output: rejected

Ques 2

Average height = 168, $\sigma = 3.9$, sample size = 25, mean = 169.5

```
t = (m - h0) / (s / sqrt(n))
p = 2 * (1 - pt(abs(t), n - 1))
```

Decision: rejected

Ques 3

```
t.test(x, y, alternative = "greater", mu = 0, paired = TRUE, var.equal = FALSE, cl = 0.95)
```

Output:

Paired t-test

t = -2.2597, df = 9, p-value = 0.9749

95% CI: -7.244834 to Inf

mean difference = -4